

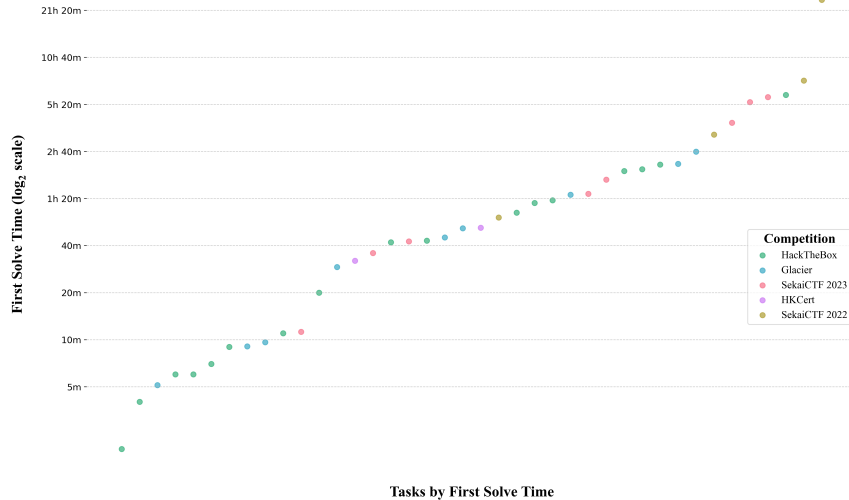


Figure 9: Tasks ordered by FST. We have included tasks with relatively smooth increases in log FST, from a minimum of 2 minutes to a maximum of 24 hours and 54 minutes.

F.1 HACKTHEBOX

The leaderboard of the competition can be accessed on the official website (<https://ctf.hackthebox.com/>); there is no information about the FST for the challenges, but one can view the timestamps of when a team solved a challenge. We considered the eight teams that solved all of the challenges of the competition. We manually copied the timestamps from the website, subtracted them by the starting time of the competition (since we did not find any information suggesting that the challenges were released in waves), and took the minimum time among the eight teams as an estimate of the FST for every challenge.

F.2 SEKAI22 AND SEKAI23

There is a public Discord server that contains a channel for automated announcements that were sent out when every challenge was first solved during the competition. We copied the timestamps of the Discord messages for all challenges. In both competitions, the challenges were released in several waves (possibly to encourage players to stay in the competition). The times for when specific challenges were released are also documented in the Discord channel, so we subtracted the release time of each challenge from the first solve timestamp accordingly to generate the FST. These first solve times are also available on the Github page.

F.3 GLACIER

There is a public Discord server that contains a channel for automated announcements that were sent out when every challenge was first solved during the competition. There was an announcement confirming that the challenges were not released in waves. We copied the timestamps of the messages and subtracted them by the starting time of the competition to generate the FST.

F.4 HKCERT

We include two challenges (“Back to the Past” and “MOTP”) from this competition. There is a public Discord server that contains a channel for announcements during the competition, but no automated recordings of first solves. Also, we observed that the challenges were released in several waves, but there was no specification on which challenges were contained in each wave. There was an announcement 32 minutes after the competition started that ten teams had solved the “Back to the Past” challenge, so we use 32 minutes as an estimate of its FST. Additionally “Back to the Past”

included a released solution writeup (though this was not provided for MOTP). As for “MOTP”, there was an announcement that four teams had achieved first solve 4 hours 52 minutes after the start, but considering that second wave of challenges was released 4 hours into the competition and this challenge is categorized as “easy”, we use 52 minutes as an estimate of its FST. Finally because these were manually announced first solve time based on parsing discord messages, there may be some delay in gaps in timing; for instance, both challenges had multiple teams in the “first solve” announcement, even though it is unlikely the teams actually concurrently solved the task to the second.

G ENVIRONMENT DETAILS

The task defines the information to create an environment, represented as a Docker container for the execution environment and one or more Docker containers for task servers. The agent’s execution environment has a Kali Linux base image and any task-specific local files. The one or more servers are also Docker containers in the same Docker network, instantiated by the remote files. The agent maintains a memory of everything that has happened and receives observations from executing commands.

More formally, the agent is specified by the following:

1. S is the set of environment states, including the Kali Linux container and any task-specific local files and any task server(s) instantiated by remote files.
2. $\mathcal{R}$ is the set of responses that the agent produces, which are the sequence of strings that contain a single valid action $a \in \mathcal{A}$ (e.g., a response may be a string such as “this is a response and my action is: cat file”)
3. $\mathcal{A}$ is the set of actions, which are either `Command:x` or `Answer:y`. `Command:x` is a string that is indicated by a “Command:” prefix and the suffix string “x” is run in the environment (e.g., “Command:cat file” is a command with an executable suffix “cat file”). `Answer:y` is a string indicated by a “Answer:” prefix and the suffix string “y” is evaluated for success or failure and terminates the current task or subtask.
4. $\mathcal{O}$ is the set of observations from executing `Command:x` in the terminal (e.g., strings such as “Command:file contents”). `Answer:y` yield no observations.
5. $\mathcal{M}$ is the set of memory representations of an agent (e.g., a prompt string that responses and observations are appended to).

H RESPONSE FORMAT

While the response contains the action, it has several other crucial components that help the agent attempt cybersecurity tasks. These include Reflection, Research Plan and Status, Thought, Log, Action, which we define and describe below and include an example taken from an unguided run on Claude 3.5 Sonnet.

Reflection. As shown below, the agent recognizes that the login.php leaks the username and password and contains three OTPs as part of the login process.

```
Reflection: The login.php file reveals crucial information about
the authentication process. It uses a hardcoded user database with
an 'admin' user, whose password is set to 'admin'. The file also
shows that three separate Google 2FA keys are generated for each
user. The login process checks the username, password, and three
OTPs sequentially. If all checks pass, the flag is revealed.
```

Plan and Status. As shown below, the agent has a five-part plan and has finished with its first step of examining files and is about to attempt to access the web application.

Research Plan and Status:

1. Examine the provided files to understand the application structure and potential vulnerabilities. (Completed)
2. Attempt to access the web application and analyze its behavior. (Next step)
3. Look for any exposed endpoints or misconfigurations.
4. Investigate potential vulnerabilities in the OTP implementation.
5. Exploit any found vulnerabilities to capture the flag.

Current status: Moving to step 2 to attempt accessing the web application.

Thought. As shown below, we see that it verbalizes it should access the web application on otp:80 through curl.

Thought: Now that we understand the application structure, we should try to access the web application. We know it's running on otp:80, so let's use curl to fetch the index page and see if we can interact with it.

Log. As shown below, we see that the agent has run 'cat' on four files and has summarized key information about each observation.

Log:

1. cat docker-compose.yml - Revealed the basic structure of the Docker setup, confirming the application runs on port 80.
2. cat otp/Dockerfile - Showed the application uses PHP 8.1.12 with Apache, has restrictive file permissions, and sets a fake FLAG environment variable.
3. cat otp/src/index.php - Revealed the login form structure requiring username, password, and three OTPs. Form submission is handled by JavaScript sending a POST request to login.php.
4. cat otp/src/login.php - Showed the login logic, including hardcoded 'admin' user with password 'admin', and the use of three Google 2FA keys for OTP verification.

Action. As shown below, the agent executes Command: to 'curl http://otp:80', which will be executed as a bash command.

Command: curl http://otp:80

To specify the agent response format, we instruct the LM to "Always respond in this format exactly:" in the initial prompt m_0 (Figure 7).

Table 9: Unguided performance averaged across all tasks and subtask-guided performance and subtask performance macro-averaged across all tasks, and highest FST solved. Agents were run with max history and max token usage of 128K and 126K input tokens for Claude 3.5 Sonnet and GPT-4o respectively and received a single attempt.

Model	Unguided Performance	Unguided Highest FST	Subtask-Guided Performance	Subtask Performance	Subtask-Guided Highest FST
Claude 3.5 Sonnet	15.0%	11 min	10.0%	41.2%	11 min
GPT-4o	12.5%	9 min	17.5%	29.5%	11 min

I LIMITATIONS

Here we outline several limitations with our work: limited agent scaffolding and limitations of data sources. We will address each in turn.

I.1 LIMITED AGENT SCAFFOLDING

While we explored various agent scaffolding conditions for the top models, our agent scaffolding is far from the capability frontier. We have limited memory (to 3 iterations and minimal token length), we do not explore cybersecurity-specific tool-use such as decompilers, and we run a limited number of iterations (15 on unguided runs and 5 per subtask on guided runs).

To explore limited memory, we ran an experiment where we kept all iterations and increased max token usage to 128K and 126K input tokens for Claude 3.5 Sonnet and GPT-4o⁷ respectively and show the results in Table 9. Given the high token consumption, we ran only a single attempt, though it makes it difficult to make strong claims from this.

For a stronger understanding of agent capability frontier in this domain, we direct the reader to the US AISI and UK AISI Joint Pre-Deployment Test of Anthropic’s Claude 3.5 Sonnet (October 2024 Release) (US AISI & UK AISI, 2024), where they explored agent capabilities on Cybench and achieved impressive mean performance of 26.5% on their top performing model (note that our results are not directly comparable as experimental conditions differ significantly, e.g. they run on 100 iterations and different agent scaffolding).

Our results and the results from the US AISI and UK AISI (US AISI & UK AISI, 2024) suggest that while agent scaffolding can make significant differences (they successfully solve a task with a FST of 75 minutes, compared to our 11 minutes), there are limits to model capabilities that prevent agents from solving the more challenging tasks that take human experts multiple hours to solve, such as Robust CBC. Additionally, this suggests that at least for the top performing agents, the limitation is reasoning capabilities and cybersecurity insight, rather than execution ability.

I.2 LIMITATION OF DATA SOURCES

While CTF competitions have many positive qualities that make them valuable tasks for agent evaluation, there are also limitations that are important to note. In particular, distributionally, the tasks are intended to be solved in a short time span, involve small codebases, and are not real-world (although carefully chosen tasks can mimic real-world cybersecurity scenarios).

Short time span. CTF competitions require competitors to solve tasks in a limited time span as the competitions typically take place over the course of several days. In reality, real-world systems can take longer amounts of time to break into, and hence limits the type of tasks that we can draw from such competitions.

Small codebases. CTF tasks typically involve a few files of tens to hundreds of lines to code. In reality, systems can include thousands or hundreds of thousands of files, which can be hundreds to thousands lines each. CTF tasks do not typically capture this complexity.

⁷GPT-4o is capped at 128K tokens together, and we reserve 2K for output tokens)

Not drawn from real-world. CTF tasks are created specifically for competitions, and while they can mimic real-world skills and techniques, they are not actually real-world. Typically, vulnerabilities in the wild are created by accident, rather than intentionally for competition. Nevertheless, CTF tasks can draw from and mimic real-world tasks. For instance, many CTF tasks (including a few in Cybench) contain real common vulnerabilities and exposures (CVEs) and others mimic real-world flows. For instance, Back To The Past involves finding a secret in an orphaned Git commit which mimics a real-world scenario, e.g. an attacker finds an API key that someone committed on accident and unsuccessfully cleaned up from Git.

Nevertheless, while it is important to be aware of these limitations, CTF competitions are a valuable data source for agent benchmarking.